

STALE, MISATTRIBUTED, OR LATE: WHAT PERSONAL FACT RETRIEVAL PUTS IN THE PROMPT

Anonymous authors

Paper under double-blind review

ABSTRACT

A language assistant that uses remembered personal facts can fail in three ways that retrieval accuracy does not separate: the prompt can carry a value that has since been revised, a value belonging to a different person with the same name, or nothing at all because retrieval finished after generation began. We formalize these as stale exposure, misattribution and lateness, and score them on the prompt rather than the answer, including on requests the store cannot answer, which a product of recall terms silently zeroes. Several memory systems remove staleness at write time by keying a fact to its subject and property and serving only the open value. We adopt that rule unchanged, claim no novelty for it, and ask what it is worth. Given correct keys it removes stale exposure for every retriever we tried, and the ranker then barely matters. The premise is the expensive part, and its two failure modes are not symmetric: a missed merge leaves a line in the prompt, a false merge deletes a value from the store. A local model assigning keys on paraphrased text reaches .987 key recall and still ends at .492 clean retrieval against a rule extractor’s .762; on LongMemEval, where keys must be discovered from open text, the mechanism stops firing altogether. The other two failures survive all of it. A posterior over the entities a name could denote cuts wrong-person exposure from .121 to .032 at no cost in recall, where filtering on the interlocutor costs 98% of facts about third parties, and neither helps on LoCoMo, where two speakers are the same string. Nothing abstains when the answer is absent unless a gate is added, and the gate that works needs a slot inventory the open corpora do not have.

1 INTRODUCTION

A user drafts a message to a colleague, Alex Chen, and the assistant offers a continuation of “the standup with Alex is at”. A useful continuation carries the time the two of them last agreed on. Three things can go wrong in the memory layer before the model writes a word: the prompt can hold 9:30am after the meeting moved to 10:45am, it can hold the time agreed with a different Alex, or the lookup can still be running when prefill starts. We call these staleness, misattribution and lateness.

Several memory systems handle the first at write time, keying a fact by its subject and the property it states, closing the old value when a new one arrives, and retrieving only what is open (Rasmussen et al., 2025; Chhikara et al., 2025; Yadav, 2026; Wang, 2026); they report that stale answers all but disappear. The rule is standard in temporal databases (Jensen & Snodgrass, 1999) and we adopt it unchanged; our question is what it is worth, and what it leaves untouched, measured on the object the model actually reads. That object is the prompt, which current evaluations do not score: LoCoMo, LongMemEval, MemoryAgentBench and Memora score the answer (Maharana et al., 2024; Wu et al., 2024; Hu et al., 2026; Uddin et al., 2026), and even a forgetting-aware score cannot see a prompt that held the current value and a superseded one at once and got lucky. We record instead, per request, whether the injected block holds the current value, a superseded value of the same slot, a same-named other person’s value, or nothing at all, and how long it took to produce.

Measured there, the write-time rule looks different. Given correct keys it removes stale exposure for every retriever we tried, so the ranker barely matters; but the keys have to come from somewhere, and the two ways of getting them wrong are not symmetric. A missed merge leaves an extra line in the prompt; a false merge deletes a correct value, silently. That is enough to put a model with .987

key recall behind a rule extractor with .797, and on LongMemEval the mechanism does not fire at all. The other two failures are untouched: wrong-person exposure survives every ranker and validity filter, and a retriever with a fixed k cannot report that it found nothing.

We contribute (i) a prompt-level decomposition and its metrics, with a two-case definition that scores requests the store cannot answer (Eq. 1) rather than zeroing them; (ii) a measurement of what write-time validity is worth, breaking its premises one at a time through key noise at known rates, key granularity, out-of-order arrival, a model assigning keys on paraphrased text, and open-domain key discovery; (iii) two serving-side rules for the failures it does not cover, each reported with the recall it costs and the corpus on which it stops working; and (iv) latency accounting, including a waiting bound whose assumptions we measure on the path it bounds. Everything runs on synthetic corpora we control and two public ones we did not, with two-level bootstrap intervals, cluster-level permutation tests and equivalence tests wherever we call two arms interchangeable.

2 RELATED WORK

Write-time validity, and why we claim none of it. Agent memories score observations by recency and relevance (Park et al., 2023), add forgetting curves (Zhong et al., 2024), page between tiers (Packer et al., 2023), link atomic notes (Xu et al., 2026), or retrieve over entity graphs and abstractions (Gutiérrez et al., 2024; Xia et al., 2026). A subset decides at write time which version is current and serves only that one, whether by a model judging contradiction (Rasmussen et al., 2025; Chhikara et al., 2025) or by a deterministic rule over (subject, relation, object) triples in a bi-temporal ledger (Yadav, 2026; Wang, 2026); others resolve conflicts at read time (Banerjee et al., 2026) or reorder on stated rather than dialogue time (Su et al., 2026).

Our rule is MemStrata’s, one key shape away, and Proposition 1 is the state invariant any of these designs maintains, itself standard for transaction-time databases (Jensen & Snodgrass, 1999). We claim no novelty for it, and use it because this paper’s question is the one those results leave open: MemStrata reports stale-fact errors falling from 15–40% to about zero and TOKI’s theorems hold by construction, but both presuppose that an update was recognized as one and attached to the right key. Sections 7.2 and 7.4 ask what happens when it was not.

What the prompt hides that the answer does not. Evaluations of agent memory mostly score the final answer: LoCoMo and LongMemEval over long histories (Maharana et al., 2024; Wu et al., 2024), MemoryAgentBench with selective forgetting as a competency (Hu et al., 2026), Memora with an accuracy penalizing reliance on invalidated memory (Uddin et al., 2026), MemConflict with conflicts built on purpose (Tao et al., 2026), and STALE by asking whether a model notices its own memory has expired (Chao et al., 2026); closest to us, MemTrace probes the retrieved context as well as the answer (Long et al., 2026). We differ in what is scored and what is controlled. An answer-level score, forgetting-aware or not, is one number over a prompt that may hold the current value, a superseded value and a third person’s value at once: Table 1 shows a store with .950 current-value recall and .703 stale exposure, which no answer-level metric distinguishes from a clean one. MemTrace keeps retrieval and generation as its stages; we cut along failure modes instead, because the three turn out to be decided in different places. STALE and MemConflict ask whether a system handles a conflict; we ask what reaches the prompt before the model is consulted, and add two failures neither covers: a name that denotes the wrong person, and a request answered after generation began. None of this work prices the memory block, which Section 7.5 finds costs three orders of magnitude more than the retrieval that filled it; work on memory cost (Zhao et al., 2025; Yuan et al., 2026; Zhou et al., 2026) and lock-free publication (McKenney & Slingwine, 1998; Gospodnetic et al., 2010) informs our serving path.

3 PROBLEM FORMULATION

Requests are indexed by t and arrive at τ_t ; $\mathcal{M}_t$ is the set of facts retained then, history included. A request carries local context c_t , a participant set P_t , a fact limit k , a character budget B and a deadline D_t .

Definition 1 (Personal fact). *A stored fact is $f = (z_f, E_f, P_f, \kappa_f, a_f, b_f, s_f)$: prompt-ready text, normalized entities, the participants of the source observation, an optional key naming a mutable*

slot, the interval $[a_f, b_f)$ during which the store treats f as active ($b_f = \infty$ until f is superseded), and the time f was last observed. The active view is $\mathcal{A}_t = \{f \in \mathcal{M}_t : a_f \leq \tau_t < b_f\}$.

That interval records the store’s state, not the truth: an extractor that misses a revision leaves an outdated fact active. A policy π maps $(\mathcal{M}_t, c_t, P_t)$ to an ordered $S_{\pi,t} \subseteq \mathcal{M}_t$ in time $L_{\pi,t}$, subject to $|S_{\pi,t}| \leq k$ and a serialized length of at most B ; it may restrict itself to $\mathcal{A}_t$ or ignore the intervals, so validity-agnostic retrievers are admissible. $R_t(f) = 1$ when f states a value for the slot asked about, for any entity whose name matches the surface form used, so R_t ignores identity; $I_t(f) = 1$ when f concerns the intended entity; $V_t(f) = 1$ when its value is still current. The target set is $\mathcal{G}_t = \{f : R_t(f) = I_t(f) = V_t(f) = 1\}$, and the policy sees none of these labels. Current-value recall is $C_{\pi,t} = \mathbf{1}\{S_{\pi,t} \cap \mathcal{G}_t \neq \emptyset\}$, stale exposure is $E_{\pi,t}^{\text{stale}} = \mathbf{1}\{\exists f \in S_{\pi,t} : R_t(f) = I_t(f) = 1, V_t(f) = 0\}$, and misattribution is $E_{\pi,t}^{\text{id}} = \mathbf{1}\{\exists f \in S_{\pi,t} : R_t(f) = 1, I_t(f) = 0\}$. With $\mathcal{G}_t = \emptyset$ there is nothing to recall, so $C_{\pi,t}$ is zero for every policy including a correct one, and a product of the three indicators scores a correct abstention and a confident wrong answer alike. The prompt is therefore clean in two cases,

$$Y_{\pi,t} = \begin{cases} C_{\pi,t}(1 - E_{\pi,t}^{\text{stale}})(1 - E_{\pi,t}^{\text{id}}), & \mathcal{G}_t \neq \emptyset, \\ 1 - \mathbf{1}\{\exists f \in S_{\pi,t} : R_t(f) = 1\}, & \mathcal{G}_t = \emptyset, \end{cases} \quad Y_{\pi,t}(D) = Y_{\pi,t} \mathbf{1}\{L_{\pi,t} \leq D\}, \quad (1)$$

which we report separately and never pool, since a policy that abstains often scores well on the second at the expense of the first. We also record the abstention rate $A_{\pi,t} = \mathbf{1}\{S_{\pi,t} = \emptyset\}$: a fixed- k retriever has $A_{\pi,t} = 0$ by construction. The indicators are not exclusive, so a prompt can hold the current value and a superseded one at once, which recall alone hides.

Suppose now that every recognized revision of a slot carries the same key κ and updates are applied in receipt order, so that committing f' with key κ at u sets $a_{f'} = u$, $b_{f'} = \infty$ and $b_f = u$ for the active predecessor, which stays as history, atomically for readers.

Proposition 1 (At most one active value per key). *Fix a key κ . If updates carrying κ are committed serially and atomically by the rule above, and at most one fact with key κ is active initially, then every reader-visible state satisfies $|\{f \in \mathcal{A} : \kappa_f = \kappa\}| \leq 1$.*

The proof is in Appendix O. The statement is narrow and says nothing about whether the surviving value is right; under out-of-order arrival the last to arrive stays active, not the last stated. Two extractor errors break it in opposite ways: a *missed merge* gives two revisions different keys, so both stay active, and a *false merge* gives one entity’s revision another entity’s key, so that entity loses its current value.

4 A REFERENCE MEMORY LAYER

To measure what enters the prompt we need a layer whose every stage is logged and whose parameters are fixed in advance. We use Personal Fact Memory (PFM), which implements the keyed-supersession rule of Section 2 with the update decision made by a key comparison at commit time rather than by a model call. Nothing here is a new mechanism; Appendix A gives every formula and constant.

An extractor turns an observation into drafts with text, entities and an optional slot key. On the controlled benchmark a rule keys a sentence naming a gazetteer entity, a slot noun and a value of that slot’s type; the public corpora of Section 7.4 have neither, so there the drafts are plain sentences and the key comes from a model. A keyed draft repeating the active value of its key is a re-observation; any other supersedes it. A keyless draft is absorbed only when an active fact sharing an entity has token Jaccard at least 0.8 and the same value-bearing tokens — without that second condition, “the standup with Alex is moved to 9:30am” absorbs “...moved to 10:45am” and the revision is lost although no key was involved. Superseded facts leave the index and stay as history, and pruning exempts the active fact of every key. Serving scores active facts by simple BM25F (Robertson et al., 2004; Robertson & Zaragoza, 2009) over text, entity and participant fields, times recency and usage factors, plus a one-hop graph term, then takes facts in score order until k or B would be exceeded; no embedding and no language model runs on this path. A background worker also publishes immutable snapshots a request can read instead of waiting, which Appendix G bounds (Proposition 2) conditional on a hand-off cost and a scheduling delay we measure rather than assume.

5 IDENTITY AND ABSTENTION ON THE SERVING PATH

Supersession decides which value of a slot is current. It says nothing about which entity the request is about, or whether the store has that entity’s value at all, and no ranker we tried does either. Both are decisions on the ranked list, and both can be made without a model call.

A name token of the request is a *mention*, ambiguous when several stored entities answer to it, as a first name does; tokens that pick out one entity are left alone. For the candidate set $\mathcal{E}_t$ of an ambiguous mention we score each e and take a posterior:

$$\psi_t(e) = w_p \mathbf{1}\{e \in P_t\} + w_g \sum_{p \in P_t} \frac{w_{pe}(\tau_t)}{\sum_v w_{pv}(\tau_t)} + w_r 2^{-(\tau_t - s_e)/T_{\text{rec}}}, \quad \pi_t(e) = \frac{e^{\psi_t(e)/T}}{\sum_{e' \in \mathcal{E}_t} e^{\psi_t(e')/T}}, \quad (2)$$

where s_e is the most recent active fact about e , $w = (1, 1, 0.25)$ and $T = 0.5$: the candidate is the interlocutor, the graph links it to the interlocutor, the store saw it recently. Retrieval keeps facts about $\arg \max_e \pi_t(e)$ and facts about no candidate. The middle term is what separates this from filtering on the conversation: a hard participant filter is the special case putting all mass on P_t , which removes wrong-person exposure on a benchmark whose facts are about the person one is talking to and deletes almost everything about third parties, whereas the graph term requires only that the subject has been discussed with the interlocutor.

A block that states nothing is the correct block when the store holds nothing. The *slot gate* recognizes the slot a request asks for from its noun and trailing cue and returns nothing unless some retrieved fact states that slot for a candidate with $\pi_t(e) \geq \tau$. The *margin gate* needs no slot inventory: it returns nothing unless the top fact scores at least τ times the shortlist mean and shares a third of the request’s content terms. The first is the stronger rule and the narrower one, so we report both on both kinds of corpus (Appendix D).

6 EXPERIMENTAL SETUP

Corpora. The controlled benchmark builds 160 revision chains per instance over (entity, slot) pairs from 36 people, 20 projects and six slot types, with zero to four revisions each and 1,000 filler messages. Queries are message prefixes such as “Just confirming, Alex standup is now at”, carrying the chain’s participant as P_t and naming people by first name, so a chain on the same slot whose person shares that first name is a wrong-person competitor: 40–50 of the 240 queries per instance are ambiguous this way. Five instances give 1,200 queries, and values unique within an instance make exposure a whole-token containment check. Two public corpora remove what the generator supplies. *LongMemEval* (Wu et al., 2024) carries staleness: its 78 knowledge-update questions each annotate two answer-bearing user turns, an earlier one stating a value later revised and a later one stating the value in force, which a local model turns into exact labels by copying the verbatim span of each; it has no slot vocabulary, so keys must be discovered. *LoCoMo* (Maharana et al., 2024) carries misattribution, because the name John belongs to a speaker in three of its ten conversations, so over one store built from all ten a block answering a question about one John with a fact from another’s conversation is a misattribution by provenance; per-conversation stores supply unanswerable requests (Appendices B, E).

Arms and statistics. Every arm ranks facts from the same store through the same greedy selection and block format with $k = 6$ and $B = 800$ ($B = 1200$ on the public corpora). BM25 uses text only; “+ validity” restricts an arm to the active facts PFM serves; “+ P_t ” appends participant names to the query and to each fact’s text; Dense is all-MiniLM-L6-v2, and Hybrid fuses both shortlists by reciprocal rank. Queries of one chain share a label set and chains of one instance share a seed, so intervals come from a two-level bootstrap over seeds then chains, and a rate of 0 or 1 gets the Wilson interval over the number of *chains* with an event, so 0 of 800 reads as a bound. Paired comparisons use a permutation test that flips the sign of a whole chain at a time, since McNemar treats one chain’s queries as independent. A difference that fails to reach significance is not equivalence, so where we call two arms interchangeable we run two one-sided tests against a margin fixed in advance at ± 0.02 clean retrieval (Schuirmann, 1987).

Store	B	Current	Stale	Wrong-person	Co-injection	Clean
Keyed	800	.998 (.993, 1.000)	.000 (.000, .005)	.121 (.094, .149)	.000 (.000, .005)	.877 (.850, .904)
Keyless	800	.950 (.923, .971)	.703 (.654, .749)	.004 (.000, .012)	.701 (.652, .746)	.247 (.210, .284)
Keyed	240	.869 (.836, .903)	.000 (.000, .005)	.000 (.000, .005)	.000 (.000, .005)	.869 (.836, .903)
Keyless	240	.823 (.791, .852)	.446 (.396, .501)	.000 (.000, .005)	.438 (.388, .493)	.385 (.331, .435)

Table 1: With and without slot keys (1,200 queries, five seeds). Co-injection: current and superseded value in one prompt. Intervals are two-level (seed, chain) bootstrap; a rate of exactly 0 gets the Wilson interval over the 800 chains, reading as a bound rather than as certainty. Shading is per column, darker for better, over the column’s range. The keyless store’s low wrong-person rate is no merit: its prompts are too full of superseded values for a competitor to fit.

Method	Clean	Current	Stale	Wrong	p50 (ms)	p99 (ms)
PFM	.877 (.850, .904)	.998	.000	.121	0.25	0.69
BM25 + validity	.774 (.737, .809)	.891	.000	.120	0.09	0.26
BM25 + validity + P_t	.873 (.847, .896)	1.000	.000	.128	0.10	0.29
Dense + validity + P_t	.836 (.803, .868)	1.000	.000	.164	18.3	93.0
Hybrid + validity + P_t	.843 (.813, .872)	1.000	.000	.157	18.2	82.4
Best without validity (BM25 + P_t)	.195 (.164, .227)	.997	.802	.007	0.18	0.46

Table 2: Retrieval arms on the same stores ($k = 6$, $B = 800$, 1,200 queries); latency from 1,200 timed runs on seed 7. All eight validity-agnostic arms fall between .155 and .195 clean (Appendix J).

7 RESULTS

7.1 SUPERSESSION, AND WHAT THE RANKER ADDS

The keyed and keyless stores receive identical drafts; the keyless store ignores keys. Supersession removes stale exposure: none of 1,200 keyed prompts over 800 chains holds a superseded value, bounding the rate at .005 rather than establishing it as zero, while 70.3% of keyless prompts do, almost always beside the current value. The benchmark revises 80% of its chains, more than a personal store is likely to; reweighting to a store where 70% of facts are never revised leaves keyed PFM at .877 and lifts the keyless store to .603, halving the gaps without changing the ordering. The keyed store’s remaining errors are all wrong-person exposures, all 145 falling on the 223 ambiguous queries, and in 94 of them the correct fact ranks first while a competitor fills a later slot — which is why Section 7.3 treats identity as a decision about the whole block rather than about the top rank.

Validity comes from the store, not the ranker. Every arm restricted to active facts has zero stale exposure, and every arm without that restriction exposes a stale value on 78–80% of queries once it receives P_t ; dense retrieval recalls the current value almost always yet has the highest stale exposure, because every version of a revised fact is equally similar to the query. Once the store and the participant are shared the ranker barely matters: BM25 + validity + P_t reaches .873 clean against PFM’s .877, a paired difference of .005 with a 90% interval of $-.008$ to $.019$, inside the $\pm .02$ margin fixed in advance, so the two are equivalent at that margin ($p_{\text{TOST}} = .038$) where the permutation test alone ($p = .55$) would show nothing. Without P_t the same arm drops to .774 and PFM leads by .103 (CI .082 to .127), so the participant name accounts for the gap; an ablation agrees, since dropping the participant field, the participant query terms or both costs .036, .067 and .122 clean retrieval while the entity field, graph, usage term, recency and term cap change nothing (Table 12). What these results support is an active-only store plus participant-aware lexical retrieval, not the rest of Section 4. Dense and hybrid with validity are worse rather than equivalent, by .042 (CI .025 to .059) and .034 (CI .018 to .050), through wrong-person exposure and not recall; and they clear a 10 ms deadline almost never, because MiniLM query encoding alone costs 18 ms at the median (Figure 2).

Serving rule	Wrong	Clean	3rd-party recall	Hard clean (answerable)	Abstained (no answer)	Abstained (main)
none	.121	.877	.985	.000	.000	.000
relative cutoff @0.7	.008	.983	.975	.908	.000	.000
participant filter	.000	.998	.020	1.000	.000	.000
identity posterior	.032	.967	.985	1.000	.000	.000
+ slot gate	.032	.967	.985	1.000	1.000	.002
margin gate	.097	.743	.815	.000	.000	.158

Table 3: Serving rules for identity and abstention. “Wrong” and “Clean” are the main benchmark; “3rd-party recall” is current-value recall where the fact concerns someone other than the interlocutor (200 queries); the last three columns are the hard-identity slice (120 answerable, 60 unanswerable) and the abstention rate on the main benchmark, the price of the gate. Stale exposure is zero in every row.

7.2 WHAT THE KEYS ARE WORTH

Proposition 1 assumes revisions of a slot share a key, so we corrupt keys after extraction. Stale exposure grows almost in proportion to the missed-merge rate p — .100 at $p=.05$, .370 at .2, .592 at .4, approaching the keyless store’s .703 — and is visible in the prompt. False merges fail silently instead: at $q=.05$, 6.1% of queries have lost their current value from the active view; at $q=.2$, 22.5% have and clean retrieval falls to .553 (Table 15). Key granularity and arrival order matter for the same reason: a key at (entity, slot) merges a person’s standup with their sync and drops current-value recall to .500, and delivering 30% of chains out of order costs .130, which ordering by stated time repairs exactly (Appendix N).

Keys from a model. A local model (qwen2:7b, greedy) rewrites every chain message of the seed-7 corpus in its own words, keeping names and values; 349 of 429 rewrites are accepted and 80 fall back to the template, and queries are unchanged. We compare the rule extractor of Section 4; the same pipeline with keys from the model; a model-resolved path where the model sees the five most similar active facts and chooses ADD, UPDATE, DELETE or NOOP — the shape of Mem0’s update decision (Chhikara et al., 2025), but our implementation of it rather than that system; and no update resolution at all. Rewriting alone costs the rule extractor .121 clean retrieval (Table 10), from .883 to .762, as key recall falls to .797 and the missed revisions surface as stale exposure. The model assigns keys better by both textbook measures, recall .987 and precision .925, and ends up worse at .492, because 40.8% of queries lose their current value outright: of its 32 key errors, 26 label a room fact as a meeting time, merging a person’s room slot into their meeting slot and deleting whichever value arrived first. The model-resolved path fails the same way, reaching .358 through deletion. Extraction quality is therefore the wrong summary: a rule that catches four revisions in five and never merges across slots beats a procedure with near-perfect recall and 7.5% false merges. The cheap guard is a type check, which would reject 30 of the model’s 32 errors — but a type check needs a type system, so Section 7.4 asks what survives on a corpus with none.

7.3 RESOLVING IDENTITY AND DECLINING TO ANSWER

A hard participant filter removes every wrong-person exposure on the main benchmark and recalls 2.0% of facts about third parties. The posterior of Eq. (2) cuts exposure from .121 to .032 and raises clean retrieval to .967 (paired difference .089, CI .071 to .110) while losing nothing: current-value recall stays at .998 overall and .985 on third-party facts, against the filter’s .020. On the hard slice, where two people share a first name and their facts share slot, noun, template and arrival day, filter and posterior both remove all exposure while a relative score cutoff leaves .092. The graph edges buy the third-party case, since the subject need only have been discussed with the interlocutor — the first place the entity graph changes an outcome, having changed none as a ranking signal. It does not replace the filter where the filter applies, leaving .032 against .000: a deployment that only ever needs facts about its interlocutor should filter, one that needs facts about the people they discuss cannot.

Construction	Merge recall	Cross-question	Current	Stale	Clean
model keys, as returned	.000	A.034	.750	A.688	C.188
linked at $\theta=0.6$	.031	C.065	.750	B.719	C.188
linked at $\theta=0.4$	.031	.123	.750	A.688	B.219
no update resolution	—	—	.719	B.719	.156
BM25	—	—	A.844	.812	.156
BM25 + recency	—	—	B.812	A.688	A.250

Table 4: LongMemEval knowledge-update: 32 labelled questions over 2,587 turns and 6,763 facts. Merge recall: questions whose two annotated answer-bearing turns got the same key, which supersession needs, capped at .344 because the extractor keys neither turn on the rest. Cross-question: keys shared across questions, i.e. false merges. Every exposure interval is about $\pm .16$ wide and no two arms are separated; the comparison rests on the first two columns, over 1,019 keyed turns. Merge recall spans .000–.031 and is unshaded: no variant merges enough to mean anything.

Abstention is easy with a schema and hard without one. No arm in the paper abstains: on the 60 unanswerable requests of the hard slice, every ranker, every validity filter and both identity rules fill the block with another person’s value of the queried type. The slot gate abstains on all 60 and costs .002 abstention where the answer is present, which presumes the inventory of six slots this benchmark has because we wrote it. The margin gate, presuming nothing, abstains on none of the 60, since the competitor’s fact is top-scoring and shares the request’s terms, exactly what a margin cannot see; and it abstains on 15.8% of the main benchmark where it should not, costing .134 clean retrieval. A temporal reranker, an entity resolver and an abstention judge run by a model at request time cost 654–912 ms each and do not win (Appendix L): the reranker halves stale exposure but loses half the current values, reaching .302 clean against .894; the judge catches two thirds of unanswerable requests where the slot gate catches all; and entity resolution, the one competitive case, reaches .931 against our posterior’s .969 for no model call.

7.4 CORPORA WE DID NOT WRITE

Everything so far runs on text our generator produced, with a slot vocabulary we chose; two public corpora remove that, one per axis. Of the 78 knowledge-update questions, 32 survive the requirement that both value spans be verbatim and distinct.

Open-domain keys do not merge, so supersession never fires. Asked to name an entity and an attribute per turn and to reuse its wording when a property recurs, the model keys 43% of turns and almost never repeats one: 1,019 keyed turns carry 891 distinct keys, and not one of the 32 questions gets the same key on both turns that state its property. A model called once per turn cannot coordinate its vocabulary, and it splits the fact differently on different calls — (“personal best time”, “time”) for one turn, (“user”, “personal best time”) for the next. We therefore add the resolution step a deployed system would run after the extractor, merging a new key into an existing one when their combined tokens reach Jaccard θ . It buys almost nothing: at $\theta = 0.6$ merge recall rises from .000 to .031 while cross-question merges rise from .034 to .065, and at $\theta = 0.4$ merge recall does not move while cross-question merges reach .123. Stale exposure is accordingly unmoved, .688–.719 for every keyed variant against .719 for no update resolution at all. The mechanism that takes stale exposure from .703 to .000 on our own benchmark does nothing here, for a reason measurable upstream of the metric: both answer-bearing turns are keyed for only 34% of questions and merged for 3%. What helps is the arm that needs no keys, BM25 reranked by recency, at .250 clean against .156.

Misattribution is worse on real text, and nothing we have removes it. Over one LoCoMo store built from all ten conversations (13,435 facts, 537 questions, 186 from a name-sharing conversation), .543 of the prompts for a question about a name-sharing speaker carry a fact from another one’s conversation, judged by provenance rather than by any value label. Filtering on the interlocutor halves that to .301 and here costs no recall, since LoCoMo’s questions are about the speakers themselves. The posterior of Eq. (2) does not apply at all: it chooses between distinct stored entities, and both Johns are the string `john`; the graph affinity we substitute cuts off-conversation lines from .371 to .237 and leaves exposure unchanged. What our benchmark puts at .121 and our posterior at .032 is .543 here and stays there — identity is the axis we leave open. Abstention fares no better: the margin

gate catches 95.0% of 1,800 unanswerable requests at $\tau = 1.5$ but abstains on 73.9% of answerable ones, dropping recall from .220 to .099, while the slot gate cannot run for want of a slot inventory (Appendix K).

7.5 RESPONSE-LEVEL EFFECTS

Prompt-level exposure does not say what the model writes, so we pass all 240 seed-7 queries through two frozen models, qwen2:7b and llama3.2:3b (4-bit, greedy), once per memory arm (Tables 18, 19). Stale prompts become stale text less often than exposure suggests, and more often for the smaller model: the keyless store exposes a superseded value on 154 prompts, repeated 45 times by the 7B model and 67 by the 3B. With keys, 92.9% of the 7B model’s completions contain the current value against 70.4% keyless (paired difference .225, CI .161 to .296); for the 3B model, .912 against .592 (difference .321, CI .246 to .402). Instructing the model that memory lines are dated helps a little and does not replace a validity filter, taking BM25 from .629 to .654 correct where BM25 over active facts reaches .842. Appendix M repeats this under key noise, out-of-order arrival, hard collisions and sampling, where the keyed–keyless gap is .229 against .225 greedy. The block also dominates end-to-end latency, taking time to first token from 325 ms to 1.22 s: the roughly 900 ms of prefill it causes is a thousand times the cost of the lexical retrieval that filled it.

8 LIMITATIONS

Data. The controlled benchmark is a small template family with unique value tokens, correct participant metadata and a known slot inventory. Keying LongMemEval costs a model call per turn, so we ingest 2,587 turns rather than the whole corpus, and requiring both value spans to be verbatim and distinct leaves 32 of 78 questions: enough to read the key diagnostics over 1,019 turns, not enough to separate two arms’ exposure rates. The dropped questions are those answered by paraphrase rather than by a span, biasing the surviving set towards quotable values — the easy case for a string-matching extractor. LoCoMo’s questions are likewise often paraphrased, so lexical retrieval recalls the gold answer rarely; it bounds exposure well and recall badly, and we use it only for the former. Neither corpus is a real user’s memory, and nothing here measures user benefit.

Claims and methods. Proposition 1 is a state invariant, not a correctness claim, and the results are conditional throughout: keyed supersession removes stale exposure *given* keys assigned to the right slot of the right entity. We do not claim that temporal correctness is decided on the write path in general. Our open-domain key assignment is also weaker than it need be, calling the model once per turn with no sight of the keys already in the store; a system that showed it candidate keys might merge more. The identity posterior resolves a mention only between distinct stored entities, and the abstention gate that works presumes a known slot inventory; neither is a solution, and our numbers say how much is at stake in each. Construction and serving are threads of one CPython process, so Proposition 2 holds only for the scheduling delay that runtime imposes; latency comes from one machine; and the response-level study uses two small quantized models and a containment judge.

9 CONCLUSION

Scoring the prompt rather than the answer separates three failures that a single accuracy number mixes, and the separation earns its keep because the three are decided in different places and yield to different fixes.

Staleness is decided when facts are written, and the rule that decides it is neither ours nor new; what our measurements add is the price of its premise. It is worth exactly as much as the key extractor that feeds it, asymmetrically: a missed merge leaves a line in the prompt, a false merge deletes a value. That puts a model with .987 key recall behind a rule extractor with .797, and on open text the keys never coincide at all, so stale exposure is what it would be with no update resolution. A near-zero stale rate is therefore a statement about an extractor and a schema as much as about a ledger.

Misattribution is decided when the block is assembled, and no validity filter touches it: filtering on the interlocutor removes it and loses almost everything about third parties, while a posterior over the entities a name could denote keeps that recall and stops working exactly where the surface forms

stop differing. Whether a request can be answered at all is a third decision a fixed- k retriever cannot make. We therefore recommend reporting what enters the prompt and not only whether the answer was right; testing the write path under its own errors before crediting the mechanism; and treating identity and abstention as first-class, because they survive every fix that staleness yields to.

ETHICS STATEMENT

The setting is personal data: messages, commitments, and details about people other than the user. Supersession keeps outdated values as history, which supports provenance, but a deployment must also honor deletion requests by removing that history and the graph edges derived from it. The reference implementation does not: `close` retires a fact from the index and leaves it in the store, and the co-occurrence edges an erased fact contributed are never withdrawn, so a store that has honored a deletion request can still route retrieval through the association the deleted fact created. Wrong-person exposure is a cross-context disclosure: a fact from a conversation with one person enters a prompt addressed to another person with the same first name, whether or not the model repeats it. We measure that cost rather than assert it: a participant filter removes the exposure and recalls 2.0% of facts about third parties, the identity posterior of Section 5 removes three quarters of it at no recall cost on distinct names, and on LoCoMo, where two speakers are literally the same name, no rule we tried removes it. Keeping participant metadata also makes a store more revealing if it leaks. None of the rules in this paper is an access-control mechanism: the identity posterior and the participant filter are ranking decisions aimed at correctness, and they reduce disclosure only as a side effect. A deployment that must not put a fact about one person into a conversation with another needs a policy that says so and is enforced where the block is assembled, not a retriever that usually ranks it lower. All experiments use synthetic, template-generated text with invented names; no human subjects, user data, or data derived from users were involved.

REPRODUCIBILITY STATEMENT

The anonymized repository contains the reference implementation, the benchmark generator, the loaders that turn LongMemEval and LoCoMo into the same benchmark format, one configuration file holding every parameter and seed, unit tests for the invariants of Section 4, one script per experiment, a script that builds every table and figure from per-query logs, the cached model calls of Sections 7.2 and 7.4, and a script that produces the anonymous archive. Model-assisted labels are cached by prompt and shipped, so the public-corpus results can be rebuilt without a GPU. Appendix A lists the parameters, Appendix B the benchmark construction and counts, Appendix E the public-corpus construction and its label audit, Appendix C the block format and prompt, and Appendix F the timing protocol, hardware, and software versions. The two public corpora are downloaded by a script in the repository; we redistribute neither.

USE OF LARGE LANGUAGE MODELS

LLM-based assistants were used partially in this project, to implement the experimental design in code and polish text. The authors specified the experiments, reviewed the code and every reported number, and take responsibility for the content.

A frozen open-weight model also appears in three other roles, each of which is part of the measurement rather than of the writing. It is the object of study in Section 7.5; it is the extractor under test in Section 7.2 and the key assigner in Section 7.4; and it produces the superseded-value labels for the LongMemEval questions, which is the one place where a model result is used as ground truth. We constrain that use to verbatim spans of an annotated turn, discard every span that fails the check, and ship the audit sheet with the reasons for each discard (Appendix E), because a label a model invented would flatter exactly the systems this paper is measuring.

REFERENCES

Pratyay Banerjee, Masud Moshtaghi, Shivashankar Subramanian, Amita Misra, and Ankit Chadha. Apex-mem: Agentic semi-structured memory with temporal reasoning for long-term conversa-

- tional ai. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16470–16489, 2026.
- Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. Stale: Can llm agents know when their memories are no longer valid? *arXiv preprint arXiv:2605.06527*, 2026.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Otis Gospodnetic, Erik Hatcher, and Michael McCandless. *Lucene in action*. Simon and Schuster, 2010.
- Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems*, 37:59532–59569, 2024.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. In *International Conference on Learning Representations*, volume 2026, pp. 156259–156291, 2026.
- Christian S Jensen and Richard T Snodgrass. Temporal data management. *IEEE Transactions on knowledge and data engineering*, 11(1):36–44, 1999.
- Xianxuan Long, Zhikai Chen, Shenglai Zeng, Shouren Wang, Kai Guo, and Jiliang Tang. Memtrace: Probing what final accuracy misses in long-term memory. *arXiv preprint arXiv:2606.17328*, 2026.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870, 2024.
- Paul E McKenney and John D Slingwine. Read-copy update: Using execution history to solve concurrency problems. In *Parallel and Distributed Computing and Systems*, volume 509518, pp. 509–518, 1998.
- Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- Stephen Robertson and Hugo Zaragoza. *The probabilistic relevance framework: BM25 and beyond*, volume 4. Now Publishers Inc, 2009.
- Stephen Robertson, Hugo Zaragoza, and Michael Taylor. Simple bm25 extension to multiple weighted fields. In *Proceedings of the thirteenth ACM international conference on Information and knowledge management*, pp. 42–49, 2004.
- Donald J Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of pharmacokinetics and biopharmaceutics*, 15(6):657–680, 1987.
- Miao Su, Yucan Guo, Zhongni Hou, Long Bai, Zixuan Li, Yufei Zhang, Guojun Yin, Wei Lin, Xiaolong Jin, Jiafeng Guo, et al. Beyond dialogue time: Temporal semantic memory for personalized llm agents. *arXiv preprint arXiv:2601.07468*, 2026.
- Zhen Tao, Jinxiang Zhao, Peng Liu, Dinghao Xi, Yanfang Chen, Wei Xu, and Zhiyu Li. Mem-conflict: Evaluating long-term memory systems under memory conflicts. *arXiv preprint arXiv:2605.20926*, 2026.

- Md Nayem Uddin, Kumar Shubham, Eduardo Blanco, Chitta Baral, and Gengyu Wang. From recall to forgetting: Benchmarking long-term memory for personalized agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 26814–26841, 2026.
- Ziming Wang. Toki: A bitemporal operator algebra for contradiction resolution in llm-agent persistent memory. *arXiv preprint arXiv:2606.06240*, 2026.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- Menglin Xia, Xuchao Zhang, Shantanu Dixit, Paramaguru Harimurugan, Rujia Wang, Victor Rühle, Robert Sim, Chetan Bansal, and Saravan Rajmohan. Memora: A harmonic memory representation balancing abstraction and specificity. In *Forty-third International Conference on Machine Learning*, 2026.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604, 2026.
- Neeraj Yadav. Temporal validity in retrieval memory: Eliminating stale-fact errors for ai agents over evolving knowledge. *arXiv preprint arXiv:2606.26511*, 2026.
- Aojie Yuan, Haiyue Zhang, and Shahin Nazarian. Agentir: A workload-adaptive cascade retrieval substrate for long-term conversational memory. *arXiv preprint arXiv:2605.25092*, 2026.
- Xinkui Zhao, Qingyu Ma, Yifan Zhang, Hengxuan Lou, Guan jie Cheng, Shuiguang Deng, and Jianwei Yin. Ame: An efficient heterogeneous agentic memory engine for smartphones. *arXiv preprint arXiv:2511.19192*, 2025.
- Wan jun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 19724–19731, 2024.
- Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. Are we ready for an agent-native memory system? *arXiv preprint arXiv:2606.24775*, 2026.

A CONFIGURATION

Component	Parameter	Value
Selection	facts per request k ; character budget B (whole block)	6; 800
	shortlist before selection	50
Query	positional decay λ ; participant weight β (additive)	0.95; 1.5
	term cap m	12
BM25F	field weights α (text, entities, participants)	1.0, 2.5, 2.0
	k_1 ; b (shared by all fields)	1.2; 0.75
Score	recency floor ε ; half-life T_{rec}	0.15; 30 days
	usage weight η ; usage half-life	0.35; 14 days
	graph weight γ	0.8
Graph	edge half-life T_g ; neighbors stored per node	30 days; 64
	damping ρ ; non-seed nodes activated; participant seeds	0.5; 12; yes
Admission	Jaccard threshold; value-token match; usage inheritance	0.8; yes; yes
Pruning	retention threshold; graph edge threshold	0.02; 0.05

Table 5: Frozen parameters, identical in every experiment. Ablations change one entry at a time.

The BM25F score of the lexical stage is

$$\text{BM25F}(q_t, f) = \sum_w q_t(w) \text{idf}(w) \frac{(k_1 + 1) \widetilde{\text{tf}}(w, f)}{\widetilde{\text{tf}}(w, f) + k_1(1 - b + b \text{dl}_f / \text{dl})}, \quad (3)$$

where $\widetilde{\text{tf}}(w, f) = \sum_F \alpha_F \text{tf}_F(w, f)$ merges the text, entity, and participant fields with the weights above, $\text{dl}_f = \sum_w \widetilde{\text{tf}}(w, f)$, and $\text{idf}(w) = \log(1 + (N - n_w + 0.5)/(n_w + 0.5))$ over the N active facts.

All experiments share one configuration file and a tagged commit. Benchmark seeds are 7, 11, 13, 17, and 19; seed 7 carries the latency, snapshot-load, response-level, and rewritten-corpus studies. Supersession orders updates by arrival unless stated otherwise; the event-time variant of Section N is the same rule with the comparison reversed when the active value was stated later.

B BENCHMARK

Chains and messages. Chains 0–11 are assigned to six pairs of people who share a first name, and both members of a pair receive a chain on the same person slot. Later chains draw an unused (entity, slot) pair at random, cycling over slot types, so they can also collide with a first name. A chain with r revisions has $r + 1$ messages with ages drawn so that later messages are more recent (between 0.3 and 27 days before the query time). Each message uses one of three templates per slot, for example “The {noun} with {entity} is moved to {value}.”, “{entity} {noun} is rescheduled to {value}.”, and “New time for the {noun} with {entity}: moved to {value}.” for meetings. When chain i satisfies $i \bmod 10 = 7$ and has a revision, all its messages use the first template (verbatim revisions). A cancelled chain ends with “The {noun} with {entity} is cancelled.” and its current value is the cancellation. Each message’s participant is the person for person slots and a fixed randomly chosen person for project slots. Filler messages come from four templates (ticket assignments, deployments, hardware maintenance, license reservations) with bot senders as participants and eight names that do not occur in chains, and twelve short chit-chat messages to random people are added.

Queries and labels. Query templates end where the value would begin, for example “Reminder — the standup with Alex is at” and “Just confirming, Alex standup is now at”. The labels of a query are the chain’s current value, its superseded values (all values, for a cancelled chain), and the current values of chains on the same slot whose person has the same first name. Values are drawn from type-specific generators and rejected if their normalized form was used before, so each value belongs to one chain. Normalization lowercases, abbreviates month names, removes digit grouping commas, and removes the space before am/pm; containment requires token boundaries, so “Dec 2” does not match “Dec 24”. Plain substring containment, which an earlier version of our code used, produces false labels whenever one value is a prefix of another.

Slices. The slices of Section N reuse the same generator. *Multi-valued*: 24 people, each with a standup and a sync chain of two revisions, 240 queries over five seeds, plus 200 filler messages. *Out of order*: the main corpus with the messages of 30% of the chains permuted in arrival order, their stated times unchanged (37–43 chains per seed). *Participants*: the main corpus with each chain assigned in turn to one of four cases, the participant being the subject, another person, a pair including the subject, or absent; queries keep the subject as P_t . *Hard identity*: 12 pairs of people who share a first name, each pair holding a standup chain with the same noun, the same template, and start times within a day, plus one query per pair about a room that nobody mentioned, whose labels are all meeting values in the corpus.

Rewritten corpus. Section 7.2 rewrites the seed-7 corpus without fillers (441 messages, 429 of them in chains). A rewrite is accepted when every name survives and every value survives, after restoring respellings with the same rules the judge uses; otherwise the template sentence stays. Key assignment is scored against the generator’s (entity, slot) keys, and the model-resolved path shows the model the five highest-scoring active facts by PFM’s own ranking.

C MEMORY BLOCK AND PROMPT

A block starts with the header [Relevant memory] and has one line per fact, – (2025-06-11, to Alex Rivera) The standup with Alex Rivera is moved to room 16C., with the date of the source observation and its participants. All arms use this format and the greedy selection of Section 4. The response-level prompt is:

Seed	Msgs	Queries	Amb. chains	Amb.	Rev.	Canc.	Verb.	Para.	$d=0$	$d=1$	$d=2$	$d=3$	$d=4$
s7	1441	240	33	43	188	14	14	80	52	81	48	34	25
s11	1437	240	32	40	196	16	13	80	44	81	54	30	31
s13	1426	240	34	48	189	6	14	80	51	70	73	23	23
s17	1465	240	31	42	196	16	13	80	44	59	66	39	32
s19	1428	240	38	50	193	21	12	80	47	69	77	25	22

Table 6: Benchmark accounting per seed (filler corpus). Queries per chain: 80 chains have two queries and 80 have one. Columns after “Amb. chains” count queries: ambiguous, revised, cancelled, verbatim-revision, paraphrased query template, and revision depth d .

```
[request 1804289383]
You are helping the user write a message to {participant}.
Continue the unfinished message with only the words that
come next.
{latest-value instruction, one arm only}{memory block}
Unfinished message to {participant}:
{query prefix}
```

The first line is a random request id that prevents the server from reusing a cached prompt prefix, so every TTFT includes a full prefill. The latest-value instruction reads: “Memory lines are dated. If two lines give different values for the same thing, use the most recent one.” The judge normalizes the completion as above and assigns one of five outcomes: *correct* (current value only), *mixed* (current value together with a superseded or other-person value), *stale*, *wrong* (other-person value only), and *other*. For a cancelled chain any form of “cancel” counts as the current value.

D IDENTITY RESOLUTION AND ABSTENTION DETAILS

Candidates and aliases. Mentions are the alphabetic, non-stopword tokens of the request. A mention is ambiguous when two or more stored entity strings contain it, and the candidate set is the union of those strings. Two candidates are *aliases* rather than rivals when one’s token set contains the other’s, which is how the extractor’s “halcyon” and “the halcyon” come to name one project; aliases are pooled into a single outcome of Eq. (2) and share its probability, so only genuinely competing names are resolved. A request whose mentions all resolve to one alias group is left untouched.

Slot detector. The gate reads the slot a request asks for from its noun where the noun is unambiguous (deadline, due date, launch, cutover, day rate, budget cap) and from its trailing cue otherwise, because the benchmark’s room and meeting slots share their nouns: “the standup with Alex is in” asks for a room and “...is at” for a time. On the 1,200 main-benchmark queries and the 180 hard-slice queries the rule agrees with the generator’s slot on every one. It is a rule for this slot inventory, which is the point of also reporting the margin gate.

Thresholds. The posterior threshold τ of the slot gate makes no difference between 0.3 and 0.5 (clean retrieval .967, abstention .002 on the main benchmark); at 0.7 it starts rejecting correct facts whose candidate is not clearly ahead, and clean retrieval falls to .712 with 25.7% abstention. The margin gate was swept at $\tau \in \{1.5, 2, 3, 5\}$ on both kinds of corpus; on the controlled benchmark every setting abstains on 15.8–17.8% of answerable requests and on none of the unanswerable ones, and Section 7.4 reports the sweep on LoCoMo.

E PUBLIC CORPORA AND THEIR LABELS

LongMemEval. We use the oracle release, which ships each question with the sessions that contain its answer, and take its 78 knowledge-update questions. Each such question annotates two user turns as answer-bearing; the later one states the value in force and the earlier one the value it replaced. To turn that into labels of the kind the exposure metrics need, we ask qwen2:7b, once per annotated turn, to copy the span of that turn which answers the question at the time it was written, at most eight words and verbatim. A question is kept only when both spans occur in their own turn

word for word after the normalization of Appendix B, are at most ten tokens, and differ from one another; the rest are dropped and listed in the audit sheet the repository ships, with the reason. We never use the dataset’s own answer string as a label, because it is often a paraphrase rather than a span, and a paraphrase cannot support a containment check.

The corpus the store ingests is every user turn of those 78 questions plus every user turn of 160 further questions drawn at random from the other five question types, which serve as distractors. We cap the distractors because assigning an open-domain key costs one model call per turn. Assistant turns are not ingested: the memory is of what the user said.

Open-domain keys. The rule extractor of Section 4 does not transfer: it requires a signal verb and an anchor, and keeps 0.29 sentences per turn on this corpus against 2.75 for a rule that keeps every sentence. The public-corpus stores therefore use the latter, and keys come from the model. For each turn the model returns an entity, an attribute and the value, with the instruction to reuse its own wording whenever the same property is stated again; the key is the normalized (entity, attribute) pair, attached to the sentence that carries the reported value. Two diagnostics then measure key quality without a key annotation: *merge recall*, the share of knowledge-update questions whose two annotated turns received the same key, which is the merge supersession needs; and the *cross-question rate*, the share of keys shared by turns belonging to different questions, which is the false merge that deletes a value.

LoCoMo. We ingest every dialogue turn of all ten conversations into one store, prefixing each turn with its speaker so the speaker is part of the record and an entity of the fact. Questions are those of the single-hop, multi-hop and temporal categories whose gold answer occurs verbatim in their own conversation; the rest are skipped and counted. Misattribution needs no value label here: each fact carries the conversation it came from, and a block that answers a question from one conversation with a fact from another whose speakers share a first name is a misattribution by provenance. Three of the ten conversations have a speaker named John. For the abstention measurement we rebuild one store per conversation and ask it both its own questions and twenty questions from each other conversation, which have no answer in it.

F TIMING PROTOCOL AND ENVIRONMENT

Hardware: Apple M2 (4 performance and 4 efficiency cores), 16 GB RAM, macOS 26.5. Software: CPython 3.11.1, PyTorch 2.10.0, sentence-transformers 5.3.0, NumPy 2.4.3. Retrieval latency is `time.perf_counter` around ranking, selection, and block formatting; the store is built before timing and no construction runs concurrently. Dense retrieval uses all-MiniLM-L6-v2 on the CPU with 4 PyTorch threads, batch size 1, and fact embeddings computed at build time (1.2 s for about 1,400 facts); query embeddings are never cached. For each of the 240 seed-7 queries and each of 5 repeats, all 15 arms run in a freshly shuffled order after a warm-up pass that also produces the quality results, and percentiles use all 1,200 runs per arm. The snapshot read is timed as a reference load of a published result. In this protocol MiniLM query encoding accounts for nearly all dense latency (median 17.6 ms of 17.9 ms). The response-level study uses ollama 0.34.0 serving qwen2:7b and llama3.2:3b (both Q4_0, fully on the GPU, context 4,096), temperature 0, seed 0, and at most 24 generated tokens; the sampling check of Section 7.5 repeats the main comparison at temperature 0.7 with seeds 1, 2 and 3. The rewriting and key-assignment study calls qwen2:7b with at most 80 tokens for a rewrite, 48 for a key, and 8 for an update decision. The public-corpus study of Section 7.4 calls the same model with at most 32 tokens for an open-domain key and 32 for a label span, and the query-time baselines of Section L with 24 for a reranking and 8 for a resolution or an abstention. Every call is cached by prompt in the run directory, so a re-run needs no GPU. Model latency on the request path is timed around the call itself, in the same loop as the lexical arms.

G SERVING UNDER SNAPSHOT LAG AND LOAD

Protocol and bound. In snapshot mode, a background worker periodically runs retrieval and publishes an immutable result by assigning one reference. A request reads that reference. Each snapshot records the store version, the request context, and when it was produced. In fresh mode, the request hands its context to a retrieval worker that is already running and waits at most $\omega_t < D_t$; on timeout

it sets a cancellation flag, which the worker checks between query terms and before scoring, and returns the latest snapshot. Workers acquire the store lock; the request thread never does.

Proposition 2 (Bound on memory-induced waiting). *Assume (i) snapshots are immutable and published by a single reference assignment, (ii) the request thread acquires no lock that construction or a retrieval worker holds, (iii) reading the reference takes at most r , (iv) handing a request to an already-running worker takes at most h , a free worker being available, and (v) the request thread runs within σ of becoming runnable, at every point on this path. Then snapshot mode adds at most $\sigma + r$ to the response path, and fresh mode adds at most $h + \omega_t + \sigma + r$, independently of how long extraction, index maintenance, graph updates, or pruning take.*

Appendix O gives the proof. Assumptions (iv) and (v) carry the weight, and both are claims about the runtime rather than about the protocol. Assumption (iv) is why workers are started once and fed through a queue: an earlier version of this code started a thread per request, and `Thread.start` blocks its caller until the new thread runs, which under interpreter-lock contention took a median of 3.0 ms and a maximum of 12.3 ms, more than the whole waiting budget and a term the bound does not name. A queue hand-off costs $7 \mu\text{s}$ at the median instead. It also takes more than one worker: with a single one, a request queues behind whatever cancelled retrieval is still draining, which is head-of-line blocking that the bound does not name either. Assumption (v) is not something we can verify with a timed sleep elsewhere in the process, so Appendix G reports the σ this runtime actually imposed on this path, which is what the bound needs. The proposition bounds waiting, not freshness or model latency: snapshot age is unbounded unless publication runs on a fixed period, and a snapshot built before the latest revision exposes the old value.

Lag. For each query we build the snapshot on the store as it was just before the last one or two messages of the query’s own chain arrived (for a one-message chain, before that message), and judge it against the current labels. Table 7 shows that a snapshot one relevant update old exposes the superseded value on every revised query and lacks the fact entirely on unrevised ones. Snapshot mode is always on time but is only as fresh as its last publication.

Load. A store of 20,000 synthetic facts (Appendix H) serves 2,000 requests while a construction thread ingests facts at up to 400 per second and prunes once per second, and a snapshot worker publishes every 50 ms. Each request reads the snapshot and then issues a fresh request with a 5 ms waiting budget, both timed from the request thread, with the garbage collector enabled and disabled. Each run ingested about 2,500 facts and completed 22–23 pruning passes.

Snapshot mode behaves as the bound describes: the reference read r stays at $2 \mu\text{s}$ at the median and $30 \mu\text{s}$ at its worst. Fresh mode is where an earlier version of this appendix reported a number that could not be compared with Proposition 2. It gave σ as the wake-up delay of a 2 ms sleep taken elsewhere in the request loop, a median of 0.5 ms and a maximum of 7 ms without the collector, and then observed fresh waiting up to 35.2 ms, which $\omega + \sigma + r$ with those values does not cover. The two quantities are not the same σ . The larger missing term was starting the retrieval worker: `Thread.start` blocks its caller until the new thread runs, which cost a median of 3.0 ms and a maximum of 12.3 ms under this load. The rest is the request thread’s own descheduling while a worker holds the interpreter lock, which a sleep timed elsewhere also understates.

Workers are therefore started once and fed through a queue, which brings the hand-off term h to $17 \mu\text{s}$ at the median and $58 \mu\text{s}$ at its worst. There are four of them rather than one, because with a single worker a request queues behind whatever cancelled retrieval is still draining: that alone took the fresh-mode fallback rate from 12% to 49%, which is head-of-line blocking the bound does not name. With the pool, the worker finishes inside the 5 ms budget on 81% of requests with the collector enabled and 88% with it disabled.

What remains is the scheduling delay the bound needs, which we now report on the path it bounds rather than from a sleep elsewhere: σ is 2.1 ms at the median, 28.5 ms at the 99th percentile and 34.8 ms at its worst with the collector enabled, and 1.8, 10.6 and 16.0 ms with it disabled. The sleep-based estimate of the same quantity is 1.0, 8.5 and 32.2 ms, which understates what the bound needs by a factor of three at the 99th percentile. Proposition 2 is a conditional statement, and what these numbers say is that on this runtime the condition it needs is a scheduling bound of about 35 ms, not the 7 ms a timed sleep suggests. The tail is set by the interpreter lock and the collector, which

are properties of the runtime rather than of the protocol; a runtime with real parallelism would be measured the same way and would need a different σ .

Lag	Slice	Current	Stale	Clean
0	revised	1.000	.000	.877
0	unrevised	.992	.000	.878
1	revised	.021	1.000	.000
1	unrevised	.000	.000	.000
2	revised	.012	.626	.006
2	unrevised	.000	.000	.000

Table 7: Snapshot lag measured in updates to the queried slot (1,200 queries, keyed PFM).

GC	σ	p50	p99	max	read p50	p99	max	fresh p50	p99	max	fallback
on	2.14	28.5	34.8		0.00	0.01	0.03	7.14	33.5	39.8	.192
off	1.85	10.6	16.0		0.00	0.01	0.01	6.85	15.6	21.0	.117

Table 8: Memory-induced waiting under concurrent construction (milliseconds; fresh waiting budget $\omega = 5$ ms). σ is the scheduling delay the bound of Proposition 2 requires, computed per request as the observed fresh waiting minus ω and the reference read, not as the wake-up delay of a sleep taken elsewhere. “read” is the snapshot reference read r . Fallback: fraction of fresh requests answered from the snapshot.

H STORE-SIZE SCALING

Synthetic facts name two entities drawn from a pool of one tenth of the store size (at least 200), with six verbs, six objects, and 40 participants, so posting lists for shared words grow with the store. This is a deliberately unfavorable distribution for term-at-a-time scoring. Each row builds a fresh store, times 300 queries after 50 warm-up queries, repeats the queries after `gc.freeze()`, and measures heap size in a separate build under `tracemalloc`. Per-observation construction cost stays near 0.14 ms across the grid. Retrieval cost follows the longest posting list a query traverses, not the fact count: with long-lived objects frozen, the median is 0.20 ms at 1,000 facts, 2.8 ms at 20,000, and 22.6 ms at 100,000. Without freezing, collector pauses inflate the tail from 3.4 ms to 21 ms at 20,000 facts and from 38 ms to 159 ms at 100,000. An earlier version of this generator gave every fact the participant “Partner k ”, so the token *partner* appeared in all n facts and every query traversed a posting list of length n ; that single shared term made retrieval five times slower at 20,000 facts. Synthetic corpora decide these numbers through their term frequencies, which is why we report the posting lists as well.

Facts	Queries	p50	p99	p50 (frozen)	p99 (frozen)	Add p50	Heap (MB)
100	300	0.03	0.04	0.03	0.04	0.025	0.4
1,000	300	0.20	0.23	0.20	0.22	0.076	2.9
5,000	300	0.70	0.77	0.70	0.75	0.132	13.4
10,000	300	1.39	1.59	1.30	1.39	0.136	26.5
20,000	300	2.92	21.3	2.77	3.37	0.143	52.9
50,000	300	9.20	62.9	9.71	11.6	0.141	137.5
100,000	300	23.0	158.9	22.6	37.6	0.142	275.0

Table 9: Fresh retrieval latency (ms) by store size on the synthetic distribution; “frozen” rows exclude long-lived objects from garbage collection. Add: per-observation construction cost. Heap: traced allocation after the build.

I WHERE LEXICAL RETRIEVAL STOPS BEING CHEAPER

PFM’s serving cost grows with the posting lists a query traverses, while a dense retriever pays a roughly constant query-encoding cost plus a scan. Figure 1 times both on the same synthetic stores

under two term distributions: facts drawn from six verbs and six objects (wide vocabulary), and facts that all share one verb and one object (narrow vocabulary), with the entity pool at one tenth of the store in both. The distributions differ in exactly the quantity that matters: at 20,000 facts the longest posting list a query traverses is 3,327 terms in the wide setting and 20,000 in the narrow one.

Under the narrow distribution PFM passes MiniLM (encoding plus an exact scan, no approximate index) at about 20,000 facts, reaching 69 ms at 50,000 against dense retrieval’s 20 ms. Under the wide distribution PFM is still the cheaper path at 50,000 facts, 15.7 ms against 17.4 ms, and the two curves meet around there. A personal store of a few thousand facts is far below either crossing, which is the regime the main text measures; a store that keeps everything a user reads, or a corpus whose facts share most of their words, is not.

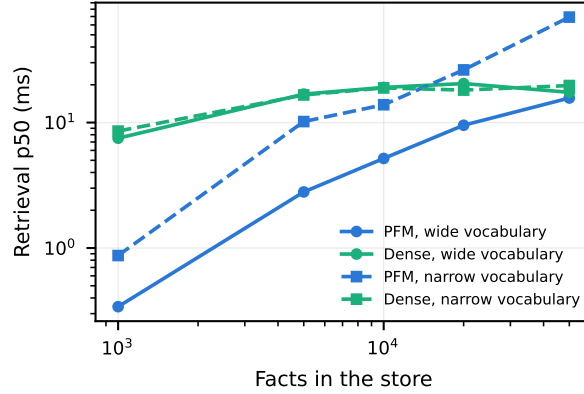


Figure 1: Median retrieval latency of PFM and a MiniLM dense retriever on the same synthetic stores (100 timed queries per point after 20 warm-up queries, long-lived objects frozen). Both axes are logarithmic.

J ADDITIONAL RESULTS

Construction	Key prec.	Key rec.	Current	Stale	Wrong	Clean	Lost
rule keys	A1.000	.797	A.896	A.154	.062	A.762	A.042
model keys	.925	A.987	.592	B.221	.083	C.492	.408
keyless	–	–	B.808	.637	A.008	.258	A.042
model updates (Mem0-shaped)	–	–	.692	C.388	.079	.358	.308
rule keys, template corpus	–	–	1.000	.000	.117	.883	.000

Table 10: Construction paths on the rewritten corpus (240 queries, seed 7). Key precision and recall are against the generator’s (entity, slot) keys. Lost: the current value is in no active fact. The last row is the same rule extractor on the template corpus and is excluded from the shading, which is otherwise as in Table 1.

K MISATTRIBUTION AND ABSTENTION ON LoCoMo

Misattribution is worse on real text, and nothing we have removes it. Over one store built from all ten conversations (13,435 facts, 537 questions, 186 of them from the three conversations with a name-sharing speaker), more than half the prompts for a question about such a speaker carry a fact from another one’s conversation, judged by provenance rather than by any value label: .543, with 37% of all served lines coming from a conversation other than the question’s own. Filtering on the interlocutor halves that to .301 and here costs no recall, because LoCoMo’s questions are about the speakers themselves. The graph affinity of Section 5 cuts off-conversation lines from .371 to .237 and leaves exposure where it was (Table 16).

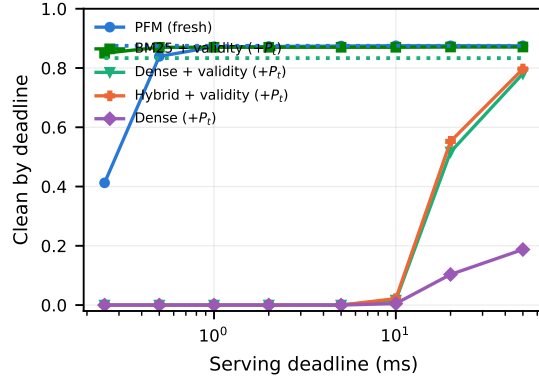


Figure 2: Clean retrieval by deadline, $Y(D)$, on seed 7. Solid lines: fresh retrieval, counting a run as successful only if it is clean and finishes within D . Dotted lines: the same ranking served from a snapshot built after the latest revision, whose read takes about $1 \mu s$; Appendix G shows what happens when the snapshot is older.

Method	Clean	Current	Stale	Wrong	p50 (ms)	p99 (ms)
PFM	.877 (.850, .904)	.998	.000	.121	0.25	0.69
BM25 + validity	.774 (.737, .809)	.891	.000	.120	0.09	0.26
BM25 + validity (+ P_t)	.873 (.847, .896)	1.000	.000	.128	0.10	0.29
Dense + validity	.834 (.802, .866)	.998	.000	.165	17.9	88.5
Dense + validity (+ P_t)	.836 (.803, .868)	1.000	.000	.164	18.3	93.0
Hybrid + validity	.807 (.773, .842)	.949	.000	.151	17.5	79.7
Hybrid + validity (+ P_t)	.843 (.813, .872)	1.000	.000	.157	18.2	82.4
BM25	.155 (.126, .186)	.852	.715	.074	0.15	0.38
BM25 (+ P_t)	.195 (.164, .227)	.997	.802	.007	0.18	0.46
BM25 + recency	.185 (.154, .215)	.877	.667	.102	0.18	0.55
BM25 + recency (+ P_t)	.193 (.162, .226)	.989	.784	.038	0.21	0.60
Dense (MiniLM)	.167 (.137, .198)	.968	.800	.131	18.5	86.6
Dense (MiniLM) (+ P_t)	.181 (.151, .212)	.989	.802	.060	19.0	97.4
Hybrid (RRF)	.165 (.136, .195)	.889	.787	.098	18.8	88.5
Hybrid (RRF) (+ P_t)	.188 (.157, .219)	.998	.802	.027	19.0	96.4

Table 11: All retrieval arms (1,200 queries; latency from seed 7). Clean intervals are two-level (seed, chain) bootstrap 95% intervals.

The posterior of Eq. (2) does not apply at all, for a reason worth stating plainly: it chooses between distinct stored entities, which works when the store holds “Alex Chen” and “Alex Rivera”, and LoCoMo’s two Johns are both the string `john`. There is no surface form to choose between, so the posterior is empty and the ranking untouched; the only evidence separating them is who each is discussed with, which the affinity term uses and which is not enough. The exposure our benchmark puts at .121 and our posterior at .032 is .543 here and stays there. Identity is the axis of the three we leave open.

Abstention on an open corpus costs more than it saves. Rebuilding one store per conversation and asking each both its own 537 questions and 1,800 questions borrowed from the other nine, the ungated ranker returns a block for almost every request it cannot answer (it abstains on 7.3%, when nothing scores at all). The margin gate does abstain, on 95.0% of the unanswerable requests at $\tau = 1.5$, but it also abstains on 73.9% of the answerable ones and drops current-value recall from .220 to .099, and raising τ only moves further along the same curve (.964 and .870 at $\tau = 2$, .983 and .968 at $\tau = 3$). No operating point on this corpus buys abstention at a recall cost worth paying. The slot gate, which does buy it on our benchmark for .002, cannot run here at all: it needs an inventory of slots, and this corpus has none. Abstention is therefore solved when the schema is known and unsolved when it is not.

Variant	Clean	Δ	Current	Stale	Wrong	Amb. clean	Amb. wrong
full	.877 (.850, .904)	+0.000	.998	.000	.121	.341	.650
no participant field	.842 (.813, .870)	-0.036	.996	.000	.154	.170	.830
no participant terms	.811 (.775, .845)	-0.067	.943	.000	.133	.211	.717
no participant seeds	.877 (.850, .904)	+0.000	.998	.000	.121	.341	.650
no participants at all	.756 (.723, .788)	-0.122	.890	.000	.138	.130	.744
no entity field	.884 (.857, .910)	+0.007	.998	.000	.114	.377	.614
no graph	.877 (.850, .904)	+0.000	.998	.000	.121	.341	.650
no usage	.877 (.850, .904)	+0.000	.998	.000	.121	.341	.650
no recency	.879 (.854, .903)	+0.002	1.000	.000	.121	.350	.650
no positional decay	.871 (.843, .899)	-0.007	.998	.000	.128	.305	.686
no term cap	.877 (.850, .904)	+0.000	.998	.000	.121	.341	.650
no usage inheritance	.877 (.850, .904)	+0.000	.998	.000	.121	.341	.650
text only dedup	.877 (.850, .904)	+0.000	.998	.000	.121	.341	.650
no keys	.247 (.210, .284)	-0.631	.950	.703	.004	.247	.022
pfm — participant filter	.998 (.993, 1.000)	+0.121	.998	.000	.000	.991	.000
bm25 validity+P — participant filter	1.000 (.995, 1.000)	+0.123	1.000	.000	.000	1.000	.000
pfm — cutoff@0.5	.898 (.873, .921)	+0.021	.998	.000	.100	.453	.538
pfm — cutoff@0.7	.983 (.973, .993)	+0.106	.992	.000	.008	.933	.045
pfm — cutoff@0.9	.866 (.835, .893)	-0.012	.866	.000	.000	.839	.000

Table 12: Leave-one-out ablation and identity baselines (1,200 queries; stale exposure is zero in every keyed row). Δ is the paired difference from full PFM with a chain-bootstrap interval. [†]Each removed separately ($\gamma = 0$, $\eta = 0$, no term cap, no participant seeds, and also no usage inheritance or text-only deduplication); none changes the outcome of any of the 1,200 queries. Relative cutoff drops facts scoring below the stated fraction of the top score.

Method	$d=0$	$d=1$	$d=2$	$d=3$	$d=4$	Benchmark mix	Rewighted
PFM (keyed)	.878	.867	.874	.887	.902	.877	.877
PFM, keyless	.794	.233	.047	.033	.023	.247	.603
BM25 + validity (+ P_t)	.857	.878	.868	.861	.910	.873	.863
Dense + validity (+ P_t)	.819	.850	.818	.848	.857	.836	.826
BM25 (+ P_t)	.983	.000	.000	.000	.000	.195	.688

Table 13: Clean retrieval by revision depth d , and aggregates under the benchmark’s own mix of depths and under a store where 70% of facts are never revised (d weights .70, .18, .07, .03, .02).

L QUERY-TIME MODEL BASELINES

Each of the three failures can also be attacked by a model call on the request path, which is what a memory that keeps every version and resolves conflicts on read must do. We put three such baselines on the same corpora, arms and metrics as the rules they would replace (480 main-benchmark queries over two seeds, 180 hard-slice queries; Table 17). A *temporal reranker* sees the dated candidate lines of a validity-agnostic retriever and names the ones in force. An *entity resolver* sees the request and the people its name could denote and picks one. An *abstention judge* sees the block and the request and says whether the block answers it.

The reranker recovers less than half of what write-time supersession gives and pays for it twice. It does cut stale exposure, from .800 to .381, but current-value recall falls from .998 to .454 as it discards lines it should have kept, so clean retrieval reaches .302 against the keyed store’s .894 — and it costs 654 ms at the median on the request path, against 0.25 ms for the whole lexical retrieval it is reranking. The abstention judge is worse: it abstains on two thirds of the unanswerable hard-slice requests, where the slot gate abstains on all of them, and abstains so freely elsewhere that clean retrieval falls from .894 to .569, for 912 ms a request.

Entity resolution is the one place the model call is competitive. It cuts wrong-person exposure from .106 to .031 and reaches .931 clean, which is what the posterior of Eq. (2) reaches at .969 for no model call at all, and it fires only on ambiguous mentions, so its median request-path cost is zero and its 99th percentile is 1,967 ms. The pattern across the three is the same: where a cheap rule exists the model call matches it at a thousand times the latency, and where no cheap rule exists the model call does not supply one either.

Method	seed 7	seed 11	seed 13	seed 17	seed 19	Mean $\pm$ sd
PFM	.875	.912	.875	.854	.871	.877 $\pm$ 0.021
BM25 + validity	.754	.812	.754	.783	.767	.774 $\pm$ 0.025
BM25 + validity ($+P_t$)	.871	.892	.867	.867	.867	.873 $\pm$ 0.011
Dense + validity ($+P_t$)	.833	.867	.812	.850	.817	.836 $\pm$ 0.023
Hybrid + validity ($+P_t$)	.842	.863	.829	.858	.825	.843 $\pm$ 0.017
BM25 ($+P_t$)	.212	.179	.208	.183	.192	.195 $\pm$ 0.015
Dense (MiniLM) ($+P_t$)	.200	.167	.179	.171	.188	.181 $\pm$ 0.013

Table 14: Clean retrieval per benchmark seed (240 queries each).

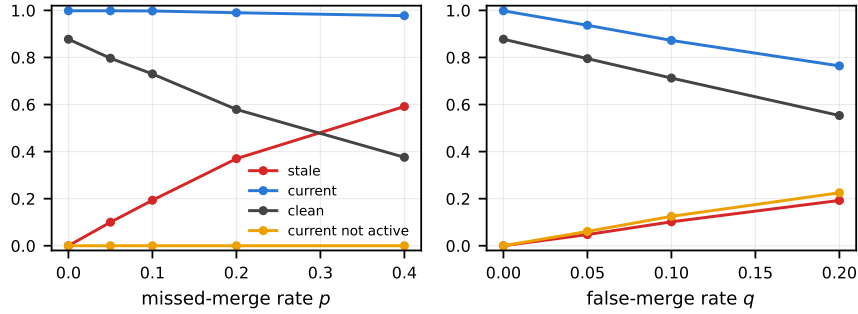


Figure 3: Keyed PFM under injected key errors (1,200 queries). Left: missed merges at rate p ($q = 0$). Right: false merges at rate q ($p = 0$). “Current not active”: the current value is held by no active fact. Table 15 gives the full $p \times q$ grid.

M RESPONSE LEVEL

Table 18 reports the 7B model on the main benchmark and Table 19 the 3B model on the same blocks. Two further studies check that those conclusions do not rest on an unbroken prompt or on greedy decoding.

Conditions that break the prompt. We repeat the measurement where the store is wrong rather than right. Under injected key noise at $p = q = .05$ the block carries a superseded value on 18.3% of requests, and the models write one on 4.2% (7B) and 10.0% (3B), so a prompt-level error rate again overstates what reaches the text, and again by less for the smaller model. Out-of-order arrival is the harder case: the block carries a superseded value on 13.3% of requests and the 7B model writes one on 13.3%, repeating essentially everything it is shown. We read the difference as a property of what the stale value looks like — a key-noise prompt usually holds the current value beside the stale one, an out-of-order prompt often holds the stale one alone — and not as a property of the models.

On the hard-identity slice the serving rules of Section 5 carry into the text. With no rule the block holds a competitor’s value on every request and the models write it on 30.6% and 33.3%. The identity posterior removes the exposure on the answerable requests, leaving it only on the unanswerable ones, and the completion rate does not move (.333 and .306), because the requests it still exposes are exactly the ones the models answer wrongly. Adding the slot gate removes both: 100% of the 7B model’s completions and 97.2% of the 3B model’s are correct, where correct on an unanswerable request means naming no value.

Sampling rather than greedy decoding. We repeat the keyed-versus-keyless comparison at temperature 0.7 with three decoding seeds, 720 completions per arm. The keyed store reaches .921 correct against .929 greedy, and the keyless store .692 against .704, so the gap is .229 sampled against .225 greedy. The comparison does not depend on the decoding rule.

p (rows), q (columns)	0	0.05	0.1	0.2
0	.877 / .000 / .000	.795 / .048 / .061	.713 / .102 / .125	.553 / .193 / .225
0.05	.797 / .100 / .000	.709 / .152 / .068	.614 / .210 / .129	.496 / .271 / .214
0.1	.730 / .193 / .000	.637 / .242 / .062	.562 / .281 / .118	.460 / .332 / .205
0.2	.579 / .370 / .000	.522 / .398 / .048	.462 / .425 / .102	.368 / .463 / .186
0.4	.376 / .592 / .000	.341 / .592 / .044	.331 / .587 / .083	.288 / .598 / .121

Table 15: Key-noise grid: clean retrieval / stale exposure / share of queries whose current value is not active (1,200 queries per cell).

Serving rule	Wrong person	name-sharing	Off-conversation lines	Current
none	.188	.543	.371	.078
participant filter	.104	.301	.087	.102
context affinity	.190	.548	.237	.050

Table 16: Misattribution on LoCoMo, judged by provenance over one store built from all ten conversations (13,435 facts, 537 questions, of which 186 come from the three conversations with a name-sharing speaker). “Wrong person”: the block holds a fact from a conversation whose speakers share the queried first name. “Off-conversation lines”: the share of all served lines that come from another conversation. “Current”: the gold answer occurs in the block, which lexical retrieval over sentence-level facts achieves rarely on this corpus, so the column bounds exposure and not recall.

N BENCHMARK SLICES

The results so far hold on a benchmark where each slot has one value, messages arrive in the order they were written, the participant of a message is the person the fact is about, and every query has an answer. Four slices break one assumption each, with the same generator, arms and metrics (Table 21).

Key granularity cuts both ways: when one person holds both a standup and a sync, a key at (entity, slot) merges them and current-value recall falls to .500, while a key at (entity, slot, noun) recalls every current value and the keyless store exposes a superseded value on every query. Coarse keys delete and fine keys stop recognizing revisions; the main benchmark hides the choice because each (entity, slot) pair has one instance. Arrival order is not statement order: delivering 30% of chains out of order costs a transaction-time store .130 clean retrieval (.877 to .747) with stale exposure .156, because a late message restating an old value closes the newer one, and Proposition 1 still holds with the wrong value active. Ordering by stated time restores the in-order numbers exactly, at the cost of trusting a stated time at commit.

The other two slices are what Section 7.3 answers. Clean retrieval holds up across subject, third-party, group and missing participants (.865 to .965), because the fact text names its subject, but the hard participant filter then recalls the current value on 2.0% of third-party facts and 1.0% of facts with no participant: identity filtering buys exposure with recall, at a price set by how often facts concern someone other than the interlocutor. And nothing abstains. With same-first-name people whose messages share slot, noun, template and arrival day, every arm exposes the competitor’s value on every query except the participant filter and a relative cutoff at 0.7 (which leaves .092); on queries whose answer is not in the store, every arm fills the block with someone else’s value of the right type, and the cutoff does not help, because the exposure comes from the top-scoring facts.

Leave-one-out ablation. Removing one component at a time (Table 12) leaves only the participant signals with a measurable effect: dropping the participant field, the participant query terms, or both costs .036, .067 and .122 clean retrieval, and on ambiguous queries clean retrieval falls from .341 to .130. The entity field, the entity graph, the usage term, recency and the term cap change nothing; removing the graph or the usage term changes no query’s outcome at all. We read that as a property of this benchmark rather than a general finding, and claim nothing for those components as ranking signals. Taken with the equivalence test above, what our retrieval results support is a store that serves only active facts together with participant-aware lexical retrieval. They do not support the rest of Section 4, and we do not ask the reader to accept it; it is described in full so the ablation has

Arm	Stale	Wrong	Clean	Abstained (no answer)	Model ms p50
bm25+P	.800	.006	.196	.000	—
llm temporal rerank	.381	.008	.302	.000	653.6
pfm	.000	.106	.894	.000	—
identity hard	.000	.031	.969	.000	—
identity hard—abstain slot@0.5	.000	.031	.969	1.000	—
llm abstain	.000	.054	.569	.667	912.1
llm entity resolve	.000	.031	.931	.000	0.01
participant filter	.000	.000	1.000	.000	—

Table 17: Query-time model calls against the serving-side rules they would replace (qwen2:7b; 480 main-benchmark queries over seeds 7 and 11, 180 hard-slice queries). Stale, wrong and clean are on the main benchmark; “abstained” is on the 60 unanswerable hard-slice requests. “Model ms” is the median wall time of the request-path model call, which is zero for the entity resolver because it fires only on an ambiguous mention (its 99th percentile is 1,967 ms). The reranker is applied to BM25 + P_t , which is the arm that needs it; every other row serves active facts.

Memory	Curr.	Stale	Wrong	Correct	Mixed	Stale	Wrong	Other	Chars	TTFT
None	—	—	—	.000	.000	.004	.000	.996	0	325
PFM	1.000	.000	.125	.929	.000	.000	.013	.058	472	1229
PFM, keyless	.908	.642	.013	.704	.008	.188	.000	.100	476	1236
BM25	.854	.692	.092	.629	.000	.254	.004	.113	468	1365
BM25 + latest-value instruction	.854	.692	.092	.654	.000	.233	.004	.108	468	1506
BM25 + validity	.887	.000	.133	.842	.000	.000	.004	.154	465	1388
Dense + validity	1.000	.000	.171	.946	.000	.000	.000	.054	476	1469

Table 18: Response level on the 240 seed-7 queries (qwen2:7b, greedy). Prompt columns: exposure in the injected block. Completion columns: judged outcome. Chars: mean block length. TTFT: median time to first token in ms, including a full prefill. The “None” arm has no memory, so its zero correct rate holds by construction.

something to remove. The graph edges do earn their place, but for a decision ranking cannot make: Section 7.3 uses them to choose which of two people a first name denotes.

Identity baselines on the main benchmark. A hard participant filter removes all wrong-person exposure and lifts clean retrieval to .998; a relative score cutoff at 0.7 reaches .983, and at 0.9 it removes the exposure along with 13% of current values. Both exploit a property of this benchmark, that the participant is the subject of the fact, which Section N removes and Section 7.3 replaces with a rule that does not need it.

O PROOFS

O.1 PROPOSITION 1

Proof. Order the commits that carry key κ by commit time $u_1 < u_2 < \dots$. Because commits are atomic with respect to readers, every reader-visible state lies between two consecutive commits, and it suffices to show the claim after each commit. Before u_1 it holds by assumption. Suppose it holds before commit j . If no fact with key κ is active, commit j activates the new fact f_j with $a_{f_j} = u_j$ and $b_{f_j} = \infty$, leaving exactly one active fact. Otherwise the induction hypothesis says the active fact f with key κ is unique, and the commit sets $b_f = u_j$ and $a_{f_j} = u_j$. Since intervals are half-open, f is inactive and f_j active at every $\tau \geq u_j$ until a later commit closes f_j , so the count stays at one. Commits for other keys and pruning (which never removes the active fact of a key) do not change activity intervals of facts with key κ . By induction the claim holds after every commit and hence in every reader-visible state. $\square$

The argument uses only the commit order. If revisions arrive out of order, the invariant still holds but the surviving value is the latest to arrive.

Memory	Curr.	Stale	Wrong	Correct	Mixed	Stale	Wrong	Other	Chars	TTFT
None	—	—	—	.000	.000	.000	.000	1.000	0	168
PFM	1.000	.000	.125	.912	.000	.000	.017	.071	472	544
PFM, keyless	.908	.642	.013	.592	.000	.279	.000	.129	476	623
BM25	.854	.692	.092	.454	.000	.408	.004	.133	468	830
BM25 + latest-value instruction	.854	.692	.092	.504	.000	.321	.004	.171	468	648
BM25 + validity	.887	.000	.133	.817	.000	.000	.004	.179	465	809
Dense + validity	1.000	.000	.171	.887	.000	.000	.008	.104	476	829

Table 19: Response level with the second model (llama3.2:3b, 4-bit, greedy) on the same 240 queries and the same blocks as Table 18.

Condition	Model	Memory	n	Prompt stale	Prompt wrong	Correct	Stale	Wrong
hard identity	llama3.2	PFM	36	.000	1.000	.694	.000	.306
hard identity	llama3.2	pfm identity	36	.000	.333	.694	.000	.306
hard identity	llama3.2	pfm identity abstain	36	.000	.000	.972	.000	.000
hard identity	qwen2	PFM	36	.000	1.000	.667	.000	.333
hard identity	qwen2	pfm identity	36	.000	.333	.667	.000	.333
hard identity	qwen2	pfm identity abstain	36	.000	.000	1.000	.000	.000
key noise p=0.05 q=0.05	llama3.2	PFM	240	.183	.087	.754	.100	.017
key noise p=0.05 q=0.05	qwen2	PFM	240	.183	.087	.825	.042	.017
out of order 0.3	llama3.2	PFM	240	.133	.108	.796	.125	.013
out of order 0.3	qwen2	PFM	240	.133	.108	.804	.133	.008
sampling	qwen2	PFM	720	.000	.125	.921	.000	.010
sampling	qwen2	pfm keyless	720	.642	.013	.692	.186	.000

Table 20: Response level where the prompt is broken on purpose. Prompt columns are exposure in the injected block; the last three are the judged completion. The sampling rows are 720 completions per arm at temperature 0.7 over three decoding seeds; every other row is greedy.

O.2 PROPOSITION 2

Proof. Snapshot mode. The request thread performs one reference read. By (ii) it never blocks on construction or a worker, by (v) it runs within σ of becoming runnable, and by (iii) the read takes at most r ; by (i) the reference always points to a complete snapshot. The added time is at most $\sigma + r$.

Fresh mode. The request thread enqueues its context, which by (iv) takes at most h and does not block on the worker, then waits on a completion event with timeout ω_t . If the worker finishes within ω_t , the wait returns and, by (v), the request resumes at most σ later, for at most $h + \omega_t + \sigma$. Otherwise the timeout expires, the thread sets the cancellation flag without blocking, and reads the snapshot as in snapshot mode, for a total of at most $h + \omega_t + \sigma + r$. In neither case does the request thread wait for a lock or an operation of construction, so the bounds do not depend on extraction, index maintenance, graph updates, or pruning. $\square$

Two things the statement does not give. It is a bound conditional on h and σ , so it is informative only with measurements of them on this path, which is what Appendix G reports; a scheduling delay measured on some other operation of the same process does not bound σ here, and an earlier version of this paper reported one that did not. And the worker itself may wait for the store lock, for example during a pruning pass; cancellation lets it exit as soon as it acquires the lock, so abandoned retrievals do not pile up behind construction, but a fresh request issued while the single worker is still draining a cancelled one queues behind it, and that queueing is inside h . The bound excludes model latency and says nothing about the age of the snapshot returned.

Condition	<i>n</i>	Current	Stale	Wrong	Clean
<i>Two instances of one slot type</i>					
key(entity,slot)	240	.500	.000	.000	.500
key(entity,slot,noun)	240	1.000	.000	.000	1.000
keyless	240	1.000	1.000	.000	.000
<i>Out-of-order arrival</i>					
in order	1200	.998	.000	.121	.877
out of order	1200	.846	.156	.103	.747
out of order, event time	1200	.998	.000	.121	.877
<i>Participant is not the subject</i>					
pfm, subject	400	.995	.000	.068	.927
pfm, third party	200	.985	.000	.120	.865
pfm, group	400	.995	.000	.030	.965
pfm, missing	200	1.000	.000	.070	.930
pfm, participant filter, subject	400	1.000	.000	.005	.995
pfm, participant filter, third party	200	.020	.000	.005	.020
pfm, participant filter, group	400	.998	.000	.000	.998
pfm, participant filter, missing	200	.010	.000	.000	.010
<i>Hard name collisions and unanswerable queries</i>					
pfm	120	1.000	.000	1.000	.000
pfm, unanswerable	60	.000	.000	1.000	.000
pfm, cutoff@0.7	120	1.000	.000	.092	.908
pfm, cutoff@0.7, unanswerable	60	.000	.000	1.000	.000
pfm, participant filter	120	1.000	.000	.000	1.000
pfm, participant filter, unanswerable	60	.000	.000	1.000	.000

Table 21: Slices that break one assumption of the main benchmark. Rows within a block share a corpus. For unanswerable queries, “wrong” counts blocks that contain some other person’s value of the queried slot type, and “clean” is the second case of Eq. (1), which requires exposing none; no arm in this table can return an empty block, so every one of them scores zero there, and Table 3 adds the arms that can.